



Full Stack Development Week 03

Topics covered:

- Multiple components
- Styling
- Data handling via HTML5's localStorage
- Login-Logout
- GitHub integration

Activity 0: Revise HTML, CSS & JavaScript (JS) || SELF-EXERCISE

In your free time, please revise the basic HTML, CSS, and JS tutorials. The reference to these can be found under **Module- Week 1 learning material**. It is crucial that you are comfortable with all the above, as there is no time to cover basic topics in this elective.

Activity 1: Layout of a page using multiple components & styling

The landing page of any website—be it the one mentioned in the assignment specification or real-life—requires a clean and professional layout.

You should always use **multiple, clearly defined components** to define the different sections of the website, i.e., header, footer, navigation, section areas, and footer.

You will need to use **CSS** or another UI library to control the styling of the various components. In this example, let's focus on **CSS**.

Using multiple components and CSS, create the following layout for a landing page of a web application.

Additional Notes:

There are various ways to style components beyond CSS, such as:

- CSS modules
- Sass
- Less
- Tailwind (built in the NextJS boilerplate)
- Chakra UI
- Shadcn UI
- Mantine UI

While we won't cover all of these in-depth, we will look at **chakra-ui**.

Note: Chakra UI recently launched version 3. However, for these examples, we will be using Chakra UI version 2 as there is more resources online for you to reference if needed. If you want to have a play using Version 3, feel free to do so.

[Chakra UI v2 Documentation](#)

[Chakra UI NextJS Pages Guide](#)

Activity 2: Style a form in React using chakra-ui

Installation:

```
npm i @chakra-ui/react@2 @emotion/react @emotion/styled framer-motion
```

Ensure you run this command inside your project directory.

Next let's add the Chakra provider to our `_app.tsx` file.

```
import * as React from "react";

// 1. import `ChakraProvider` component
import { ChakraProvider } from "@chakra-ui/react";

function App() {
  // 2. Wrap ChakraProvider at the root of your app
  return (
    <ChakraProvider>
      <TheRestOfYourApplication />
    </ChakraProvider>
  );
}
```

Official documentation:

- [Getting Started](#)
- [Form Controls Overview](#)

Using the above documentation, create a **form styled via chakra-ui**.

Time to Ponder:

Do you know of any other styling libraries used in React web applications?

Activity 3: Store form data via HTML5's localStorage

Note: Successful completion of this activity will help with the **registration feature** in Assessment 1.

HTML5 introduced **localStorage**, which allows data to be stored in the browser's memory persistently (even after closing the browser window).

Steps:

1. Modify the form from Activity 2 to store the form data in **localStorage**.
2. Verify the data is stored using Chrome Developer Tools:
 - Open **Developer Tools** → **Application** → **Storage** → **localStorage**.

Activity 4: Login-Logout System

Note: Successful completion of this activity will help with the **login-logout feature** in Assessment 1.

Workflow:

1. **React application starts**
2. Navigation bar displays a **Login link**
3. **Login form** appears with built-in validation
4. User enters **correct credentials** → navigates to the home page
5. **Logout link** appears in the navigation bar
6. User can **log out successfully**

Concepts Used:

- Chakra UI form
- Data storage via localStorage
- Functional components & hooks
- Routing using NextJS
- Conditional rendering

Understanding Routing in Next.js

Next.js provides a file-system based routing mechanism that's both powerful and intuitive. Here are the key concepts:

- **Pages Directory:** Files in the **pages** directory automatically become routes
 - **pages/index.tsx** → **/**
 - **pages/about.tsx** → **/about**
 - **pages/posts/[id].tsx** → **/posts/1**, **/posts/2**, etc.
- **Dynamic Routes:** Use square brackets **[]** for dynamic segments
 - Perfect for content that depends on external data
 - Example: **pages/users/[userId].tsx** matches **/users/123**
- **Nested Routes:** Create nested folder structures for complex routing
 - **pages/blog/[slug]/comments.tsx** → **/blog/post-1/comments**
- **Link Component:** Use **<Link>** for client-side navigation

```
import Link from "next/link";

<Link href="/about">About Page</Link>;
```

Official Documentation:

- [Pages Router Overview](#)

- [Dynamic Routes](#)
 - [Linking and Navigating](#)
-

Activity 5: Bringing Git to Complete the Picture

This exercise assumes you have joined the **rmit-fsd-2025-s1 GitHub organisation** and have Git installed.

Info: If you have not yet joined the organisation, or have not received a GitHub invitation, please contact matt.hayward@rmit.edu.au.

Steps:

1. Create a Next.js project:

```
npx create-next-app@latest s1234567-test
```

(Replace **s1234567** with your student ID.)

When prompted, select:

- TypeScript → Yes
- ESLint → Yes
- Tailwind CSS → Yes
- src/ directory → Yes
- App Router → No
- Import alias → No

2. Open the project folder in **Visual Studio Code**.

3. In VS Code, open the Source Control tab (Ctrl+Shift+G or click the branch icon).

4. Click "Publish to GitHub" to create a new Git repository.

5. Type an initial commit message like "Initial commit" in the message field.

6. Click the checkmark icon (✓) above the message to commit the changes.

7. Click "Publish Branch" to push to GitHub.

- Select "Private repository" when prompted
- Choose a repository name
- Click "Publish"

You can find more instructions on getting started with Git in Visual Studio Code at the following URL:

<https://code.visualstudio.com/docs/sourcecontrol/intro-to-git>.

Committing and Pushing Changes:

1. Make changes to your code files.
2. In VS Code's Source Control tab:
 - Changed files appear under "Changes"
 - Click the + icon next to files to stage them
 - Type a descriptive commit message
 - Click the checkmark (✓) to commit
3. Push commits to GitHub:
 - Click the "..." menu in Source Control
 - Select "Push" or use Ctrl+Shift+P
 - Or click the sync icon in the status bar
4. Best practices for commits:
 - Write clear commit messages
 - Commit related changes together
 - Commit often
 - Don't commit broken code

Example commit messages:

- "Add login form component"
- "Fix navigation bug in header"
- "Update styling for mobile responsiveness"
- "Implement user authentication"
- "Add form validation to registration"

Using Git from the Command Line:

1. Initialize a new Git repository:

```
git init
```

2. Add your GitHub repository as a remote:

```
git remote add origin https://github.com/username/repository.git
```

3. Stage all files:

```
git add .
```

4. Create initial commit:

```
git commit -m "Initial commit"
```

5. Push to GitHub:

```
git push -u origin main
```

For ongoing changes:

1. Check status of changes:

```
git status
```

2. Stage modified files:

```
git add <filename>  
# Or stage all changes:  
git add .
```

3. Commit changes:

```
git commit -m "Your commit message"
```

4. Push to GitHub:

```
git push
```

Common Git commands:

- `git pull` - Get latest changes from remote
- `git branch` - List branches
- `git checkout -b branch-name` - Create and switch to new branch
- `git merge branch-name` - Merge branch into current branch
- `git log` - View commit history